

Reproducing Executions

Tests Created

- Selenium: `selenium_tests/test_gui_assignment.py`
- Pculix: `oculix_tests/gui_assignment.sikuli/gui_assignment.py`

Usability

Selenium is easier to use for normal website testing because it interacts with the browser. We can select elements by IDs, CSS selectors, labels, links, and button names.

OculiX is better when testing the screen as a human sees it. It can click visible text, image regions. This is useful when the target does not expose stable selectors, or when the goal is to test the visual result rather than the HTML structure. However, it takes more tuning because the script depends on window size, OCR quality, and whether the relevant text is visible on screen.

Ease of Getting Started

Selenium IDE is very easy because it can record browser actions and play them back.

OculiX is highly intuitive because when you point at what is visible and automate it. However, when writing the first script, the environment configuration phase proves to be critical and sensitive. Specifically, the browser window must be active in the foreground; the OCR engine must be capable of accurately recognizing on-screen text; furthermore, visual testing typically requires that the screen dimensions remain consistent.

Documentation

Selenium has mature documentation and a large community. The official Selenium IDE documentation explains recording and playback.

OculiX documentation is also clear: visual automation based on what appears on screen, with OCR and image recognition.

Features

Selenium strengths:

- DOM-aware locators such as ID, CSS, XPath, link text, and ARIA-style selectors.
- Explicit waits for dynamic pages.
- Headless execution for CI.
- Browser profile support for authenticated sessions.
- Fast execution because it does not need to inspect screen pixels for every action.

OculiX strengths:

- Visual automation independent of DOM structure.
- OCR and image matching for testing what the user sees.

- Can automate desktop apps, browser pages, remote screens, and other GUIs.
- Useful for canvas-heavy, image-heavy, or inaccessible interfaces.

Speed

Selenium is faster. It sends commands directly to the browser and waits for specific conditions. OculiX is slower because it must inspect the screen, run OCR or image matching.

Limitations

Selenium limitations:

- It works best on browser-based apps with accessible DOM elements.
- It may miss visual-only bugs, such as overlapping text or wrong colors, unless screenshots or visual checks are added.
- Highly dynamic selectors can make tests brittle if the page does not expose stable IDs or labels.
- University SSO and multi-factor authentication are not ideal to automate directly, so the UD test should reuse an authenticated profile.

OculiX limitations:

- OCR can misread text, especially at small font sizes or unusual contrast.
- Tests can break when browser zoom, resolution, theme, or layout changes.
- It is harder to run reliably in headless CI because it needs a real or virtual screen.
- Assertions are less precise than DOM assertions unless image anchors or carefully controlled screen regions are used.

Recommendation

For normal task, Selenium is the better default. It is faster, easier to run repeatedly, and produces clearer failures. For black-box testing, OculiX is valuable. In a real testing strategy, we would use Selenium for most web workflows and add OculiX for visual-only or cross-application scenarios.